

# Alocação Dinâmica de Memória

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz  
marcosleipnitz@ic.ufrj.br

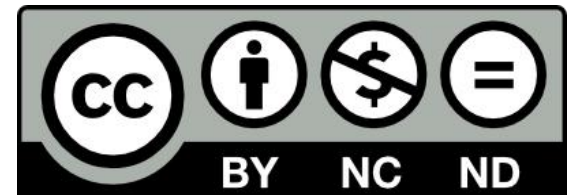


**UFRJ**

UNIVERSIDADE FEDERAL  
DO RIO DE JANEIRO



INSTITUTO DE  
COMPUTAÇÃO  
UFRJ



- Alocação dinâmica de memória
  - Vetores de ponteiros
  - Alocação de Arrays
  - Alocação de Estruturas
- Exemplos

- Sempre que escrevemos um programa, é preciso **reservar espaço** para as informações que serão processadas.
- Para isso utilizamos as **variáveis**
  - Uma variável é uma posição de memória que armazena uma informação que pode ser modificada pelo programa
  - Ela deve ser definida **antes de ser usada**

# Alocação Dinâmica de Memória

- Infelizmente, nem sempre é possível saber, em **tempo de execução**, o quanto de memória um programa irá precisar.
- **Exemplo:** Faça um programa para cadastrar o preço de **N** produtos, em que **N** é um valor informado pelo usuário.

```
int N, i;  
double produtos[N];
```

**Errado!** Não sabemos o valor de **N**

```
int N, i;  
  
scanf ("%d", &N)  
  
double produtos[N];
```

Funciona, mas não é seguro. A disponibilidade de **Variable Length Arrays (VLAs)** depende da versão do padrão C (implementação do compilador).

- A **alocação dinâmica de memória** permite ao programador criar “variáveis” em **tempo de execução**, ou seja, alocar memória para novas variáveis quando o programa está sendo executado, e não apenas quando se está escrevendo o programa.
  - A quantidade de memória é alocada **sob demanda**, ou seja, quando o programa precisa (menos desperdício de memória).
  - Espaço é reservado até **liberação explícita**.
  - Depois de liberado, estará disponível para outros usos e não pode mais ser acessado.
  - Espaço alocado e não liberado explicitamente é **automaticamente** liberado ao final da execução.

A linguagem C oferece apenas **4 funções** para o sistema de alocação dinâmica de memória, disponíveis na **stdlib.h**:

- **malloc**
- **calloc**
- **realloc**
- **free**

As funções pedem ao **sistema operacional** para separar **blocos** de memória (na *heap*) e devolvem ao programa que pediu o endereço inicial deste bloco.

## `void *malloc(size_t size)`

- Reserva **size** bytes de espaço (bloco) na memória (não inicializa o espaço alocado).
- Retorna um ponteiro para o primeiro byte reservado ou **NULL**. no caso de algum erro ocorrer (e.g., não há memória suficiente).
- O ponteiro **void\*** pode ser atribuído a qualquer tipo de ponteiro via **type cast**.

Alocar **1000 bytes** de memória livre:

```
char *p;  
p = (char *) malloc(1000);
```

Alocar espaço para **50 inteiros**:

```
int *p;  
p = (int *) malloc(50*sizeof(int));
```

# Alocação Dinâmica de Memória

```
void *malloc(size_t size)
```

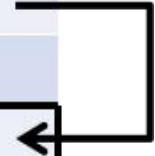
```
int *p = NULL;  
p = (int*) malloc(5*sizeof(int));
```

| Memória |          |          |
|---------|----------|----------|
| posição | variável | conteúdo |
| 119     |          |          |
| 120     |          |          |
| 121     | int *p   | NULL     |
| 122     |          |          |
| 123     |          |          |
| 124     |          |          |
| 125     |          |          |
| 126     |          |          |
| 127     |          |          |
| 128     |          |          |

Alocando 5  
posições de  
memória em int \*p



| Memória |          |          |
|---------|----------|----------|
| posição | variável | conteúdo |
| 119     |          |          |
| 120     |          |          |
| 121     | int *p   | 123      |
| 122     |          |          |
| 123     | p[0]     | ?        |
| 124     | p[1]     | ?        |
| 125     | p[2]     | ?        |
| 126     | p[3]     | ?        |
| 127     | p[4]     | ?        |
| 128     |          |          |



# Alocação Dinâmica de Memória

- Se não houver memória suficiente para alocar a memória requisitada, a função **malloc** retorna um ponteiro nulo.

```
int main(){
    int *p;
    p = (int *) malloc(5*sizeof(int));
    if(p == NULL){
        printf("Erro: Memoria Insuficiente!\n");
        system("pause");
        exit(1);
    }
    int i;
    for (i=0; i<5; i++){
        printf("Digite o valor da posicao %d: ", i);
        scanf("%d", &p[i]);
    }

    return 0;
}
```

Obs.: o comando **system("pause")** é específico do Windows (stdio.h).

Obs.: a função **exit()** encerra o programa **inteiro** imediatamente.

**void \*calloc(size\_t num, size\_t size)**

- Reserva espaço na memória para um vetor de **num** itens.
- Cada item tem tamanho **size** bytes e todos bits do espaço são **inicializados com 0**.
- Retorna um ponteiro para o espaço reservado ou **NULL** no caso de algum erro ocorrer.
- O ponteiro **void\*** pode ser atribuído a qualquer tipo de ponteiro via **type cast**.
- A função **calloc** faz o mesmo que a função **malloc**. A diferença é que passamos a quantidade de posições a serem alocadas e o tamanho do tipo de dado alocado como **parâmetros distintos** da função e o **conteúdo é inicializado com 0**.

# Alocação Dinâmica de Memória

**void \*calloc(size\_t num, size\_t size)**

```
int main() {
    //alocação com malloc
    int *p;
    p = (int *) malloc(50*sizeof(int));
    if(p == NULL) {
        printf("Erro: Memoria Insuficiente!\n");
    }
    //alocação com calloc
    int *p1;
    p1 = (int *) calloc(50, sizeof(int));
    if(p1 == NULL) {
        printf("Erro: Memoria Insuficiente!\n");
    }

    return 0;
}
```

**void \*realloc(void \*pont, size\_t size)**

- Altera o tamanho do bloco na memória apontado por **pont** para o tamanho especificado por **size**. O valor de **size** pode ser maior ou menor que o original.
- O conteúdo do bloco **será mantido** até um tamanho igual ao menor dos dois tamanhos (novo e antigo).
- Se o novo tamanho requerer movimentação de dados, o espaço reservado anteriormente é **liberado**.
- Se o novo tamanho do bloco for maior, o conteúdo da porção de memória reservada a mais ficará com um **valor indefinido**.
- Se o tamanho **size** for igual a 0 e **pont** não é um ponteiro nulo, o bloco previamente reservado é liberado.

## `void *realloc(void *pont, size_t size)`

- Um ponteiro para o bloco é devolvido porque **realloc** pode precisar **mover** o bloco para aumentar seu tamanho.
- Se isso ocorrer, o conteúdo do bloco antigo é **copiado** para o novo bloco, e nenhuma informação é perdida.

### Observações:

- Se **\*pont** for nulo, aloca **size** bytes e devolve um ponteiro (igual **malloc**)
- Se **size** é zero, a memória apontada por **\*pont** é liberada.
- Se não houver memória suficiente para a alocação, um ponteiro nulo é devolvido e o bloco original é deixado inalterado.

```
int main() {
    int i;
    int *p = malloc(5*sizeof(int));
    for (i = 0; i < 5; i++) {
        p[i] = i+1;
    }
    for (i = 0; i < 5; i++) {
        printf("%d\n", p[i]);
    }
    printf("\n");
    //Diminui o tamanho do array
    p = realloc(p, 3*sizeof(int));
    for (i = 0; i < 3; i++) {
        printf("%d\n", p[i]);
    }
    printf("\n");
    //Aumenta o tamanho do array
    p = realloc(p, 10*sizeof(int));
    for (i = 0; i < 10; i++) {
        printf("%d\n", p[i]);
    }

    return 0;
}
```

## `void free(void *pont)`

- Libera o espaço de memória apontado por **pont**.
- Diferente das variáveis definidas durante a escrita do programa, as variáveis alocadas dinamicamente **não são liberadas automaticamente** pelo programa.
- Quando alocamos memória dinamicamente é necessário liberar quando ela não for mais necessária (evitar *memory leak*).
- Assim, para liberar a memória, basta passar como parâmetro para a função **free** o ponteiro que aponta para o início da memória a ser desalocada.
- **Como o programa sabe quantos bytes devem ser liberados?**
  - Quando se aloca a memória, o programa guarda o número de bytes alocados numa "tabela de alocação" interna.

# Alocação Dinâmica de Memória

## void free(void \*pont)

```
int main(){
    int *p,i;
    p = (int *) malloc(50*sizeof(int));
    if(p == NULL){
        printf("Erro: Memoria Insuficiente!\n");
        system("pause");
        exit(1);
    }
    for (i = 0; i < 50; i++){
        p[i] = i+1;
    }
    for (i = 0; i < 50; i++){
        printf("%d\n",p[i]);
    }
    //libera a memória alocada
    free(p);

    return 0;
}
```



"Hey! Your application has a memory leak."

## Exemplo: uso de `calloc` e `free`

```
#include <stdio.h>
#include <stdlib.h>
int main ()
{
    float *v;
    int i, tam;
    printf ("Qual o tamanho do vetor? ");
    scanf ("%d" , &tam);
    v = (float *) calloc (tam, sizeof (float));
    if (!v)
    {
        printf ("Erro alocação memória");
        return -1;
    }
    for (i=0; i<tam; i++)
    {
        printf ("Elemento %d? " , i) ;
        scanf ("%f", v+i);
    }
    for (i=0; i<tam; i++)
        printf ("v[%d]=%.2f\n",i,*(v+i));
    free (v);
    return 0;
}
```

# Vetores de Ponteiros

- Uma vez que ponteiros são variáveis, podemos criar **vetores de ponteiros**, permitindo representar estruturas de dados bidimensionais.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main ()
{
    int tam,i;
    char *g[3];
    tam = sizeof(char);
    g[0] = (char *)malloc(13*tam);
    strcpy(g[0], "Computacao I");
    g[1] = (char *)malloc(23*tam);
    strcpy(g[1], "Sistemas de Informacao");
    g[2] = (char *)malloc(14*tam);
    strcpy(g[2], "Computacao II");
    for (i=0;i<3;i++)
        free(g[i]);
    return 0;
}
```

- Para armazenar um array o compilador C calcula o tamanho, em bytes, necessário e reserva **posições sequenciais** na memória.
- Note que isso é muito parecido com **alocação dinâmica**
- Existe uma ligação muito forte entre ponteiros e arrays: o nome do array é apenas um ponteiro que aponta para o primeiro elemento.

# Alocação de Arrays

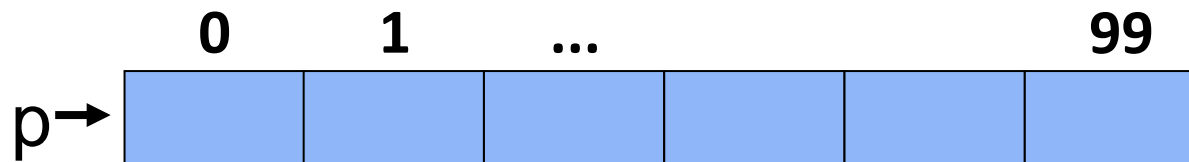
- Ao alocarmos memória dinamicamente estamos, na verdade, alocando um array.
  - O ponteiro aponta para uma área de memória que é endereçada de forma linear.

```
int *p;  
int i, N = 100;
```

Vetores (arrays unidimensionais) podem ser facilmente manipulados com ponteiros.

```
p = (int *) malloc(N*sizeof(int));
```

```
for (i = 0; i < N; i++)  
    scanf("%d", &p[i]);
```



Endereços de memória

- Para alocarmos memória para **arrays com mais de uma dimensão**, é necessário mapear o espaço multidimensional para uma única dimensão.
- No caso das matrizes (arrays bidimensionais), é necessário mapear o endereço de cada elemento na matriz, que é definido por um par **(linha, coluna)**, em um endereço linear.

- Considere, por exemplo, uma matriz chamada matriz de tamanho **LIN** x **COL**. Podemos usar uma solução que mapeie a matriz, que é um objeto de **duas dimensões**, em um vetor que tem apenas **uma dimensão**.
- Neste caso, o programa deve fazer a **translação de endereços** toda vez que precisar ler ou escrever na matriz. A expressão **matriz+(i\*COL+j)** calcula a posição do elemento **matriz[i][j]** a partir do primeiro elemento da matriz que está no endereço inicial **matriz**.
- Solução não muito boa :(

# Alocação de Arrays

```
#include <stdio.h>
#include <stdlib.h>
#define LIN 3
#define COL 2
int main ()
{
    int * matriz1 = (int *) malloc (LIN * COL * sizeof (int)), i, j;
    if (!matriz1)
    {
        printf ("Erro de alocação de memória\n");
        return -1;
    }
    for (i=0; i<LIN; i++)
        for (j=0; j<COL; j++)
        {
            printf("Matriz[%d][%d]: ",i,j);
            scanf ("%d", matriz1 + ((i * COL) + j));
        }
    for (i=0; i<LIN; i++)
        for (j=0; j<COL; j++)
        {
            printf ("%d] ", *(matriz1 + ((i * COL) + j)));
            printf("\n");
        }
    free(matriz1);
    return 0;
}
```

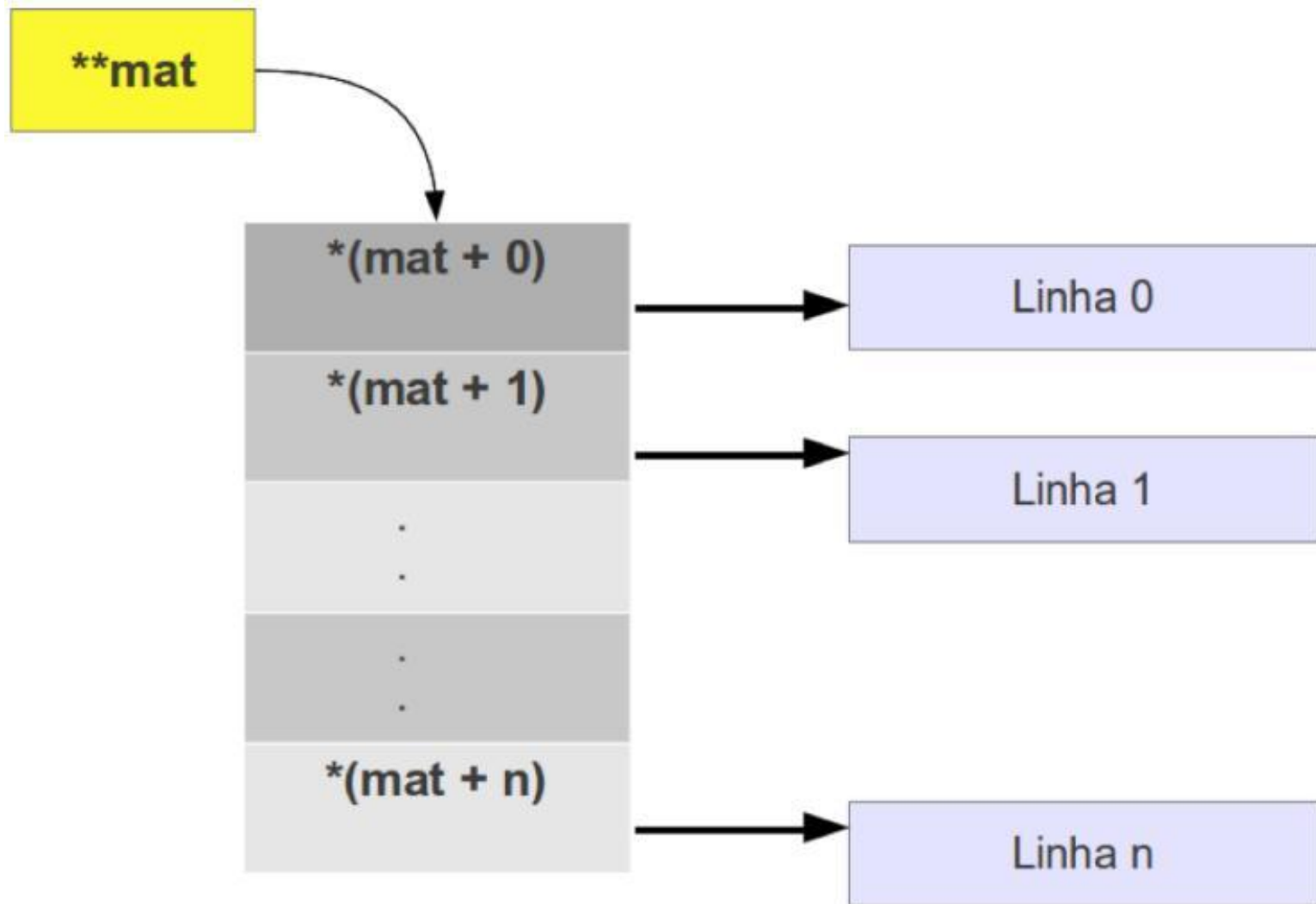
- Uma possibilidade mais interessante é utilizar **vetores de ponteiros**. Esta não é a notação ideal, mas é um passo na direção da notação mais efetiva.
- Neste caso, cada linha da matriz é apontada por um ponteiro armazenado no vetor de ponteiros.

# Alocação de Arrays

```
int *matriz2[LIN], i, j;
for (i=0; i<LIN; i++)
{
    matriz2[i] = (int *) malloc (COL * sizeof (int));
    if (!matriz2[i])
    {
        printf ("Erro de alocação de memória\n");
        return -1;
    }
}
for (i=0; i<LIN; i++)
    for (j=0; j<COL; j++)
    {
        printf ("Matriz[%d][%d]: ", i, j);
        scanf ("%d", matriz2[i] + j);
    }
for (i=0; i<LIN; i++)
{
    for (j=0; j<COL; j++)
        printf ("[%d] ", *(matriz2[i] + j));
    printf("\n");
}
```

- No exemplo anterior, o número de linhas da matriz é **fixa** e há uma **mistura de notações** de ponteiros com matrizes.
- Vamos considerar um exemplo onde tanto o número de linhas como o de colunas é desconhecido e criar um **vetor de ponteiros** que irá armazenar o **endereço inicial de cada linha**.

# Alocação de Arrays



# Alocação de Arrays

```
int i, j, L, C;
int **matriz3;
puts("Informe o numero de linhas e colunas");
scanf("%d %d", &L, &C);
matriz3 = (int **) malloc(L * sizeof(int *));
if(!matriz3)
{
    printf ("Erro de alocação de memória\n");
    return -1;
}
for (i=0; i<L; i++)
{
    *(matriz3 + i) = (int *) malloc (C * sizeof (int));
    if (! (*(matriz3 + i)))
    {
        printf ("Erro de alocação de memória\n");
        return -1;
    }
}
...
```

# Alocação de Arrays

```
...  
for (i=0; i<L; i++)  
{  
    for (j=0; j<C; j++)  
    {  
        printf ("Matriz[%d][%d]: ", i, j);  
        scanf ("%d", *(matriz3 + i) + j);  
    }  
}  
for (i=0; i<L; i++)  
{  
    for (j=0; j<C; j++)  
        printf ("[%d] ", (*(matriz3 + i) + j));  
    printf("\n");  
}
```

**Esta é a melhor opção:** notação mais **genérica** que permite alocação dinâmica das duas dimensões da matriz sem precisar usar VLAs.

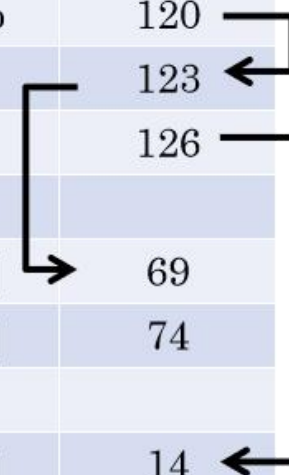
# Alocação de Arrays

- Portanto, alocamos arrays com mais de uma dimensão utilizamos o conceito de **ponteiro para ponteiro**.
- Para cada nível do ponteiro, fazemos a alocação de uma dimensão do array.

```
int **p; //2 "*" = 2 níveis = 2 dimensões
int i, j, N = 2;
p = (int**) malloc(N*sizeof(int*));

for (i = 0; i < N; i++){
    p[i] = (int *)malloc(N*sizeof(int));
    for (j = 0; j < N; j++)
        scanf("%d", &p[i][j]);
}
```

| Memória |          |          |
|---------|----------|----------|
| posição | variável | conteúdo |
| 119     | int **p  | 120      |
| 120     | p[0]     | 123      |
| 121     | p[1]     | 126      |
| 122     |          |          |
| 123     | p[0][0]  | 69       |
| 124     | p[0][1]  | 74       |
| 125     |          |          |
| 126     | p[1][0]  | 14       |
| 127     | p[1][1]  | 31       |
| 128     |          |          |



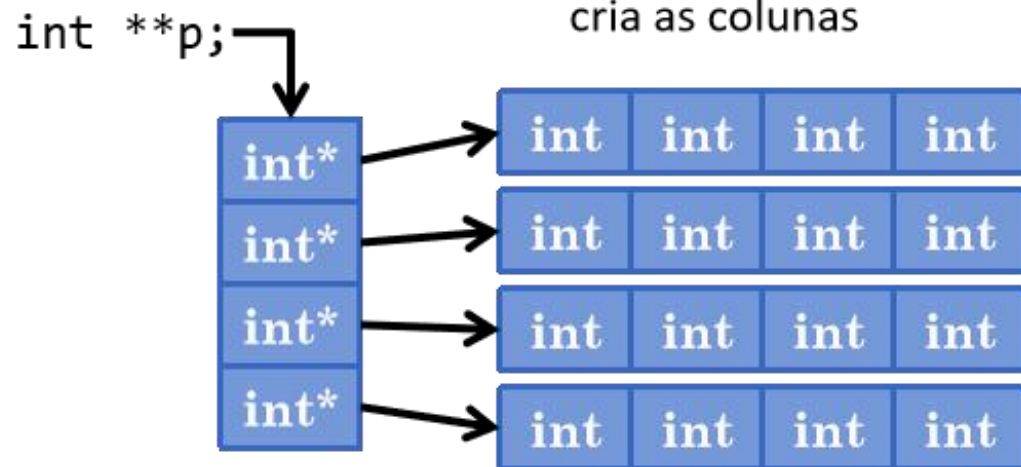
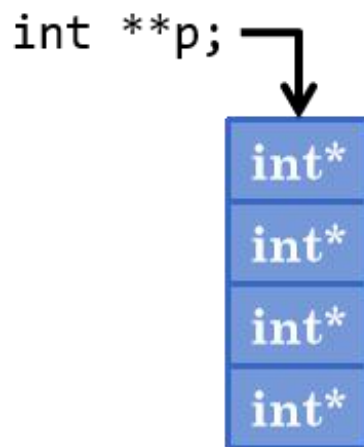
# Alocação de Arrays

- Para alocarmos arrays com mais de uma dimensão, utilizamos o conceito de “**ponteiro para ponteiro**”.
- Para cada nível do ponteiro, fazemos a alocação de uma dimensão do array.

```
p = (int**) malloc(N*sizeof(int*));
```

```
for (i = 0; i < N; i++){  
    p[i] = (int *)malloc(N*sizeof(int));  
}
```

1º malloc:  
cria as linhas



- Diferente dos arrays de uma dimensão, para **liberar a memória** de um array com mais de uma dimensão é preciso liberar a memória alocada em cada uma de suas dimensões, na ordem inversa em que foi alocada.

```
int **p; //2 "*" = 2 níveis = 2 dimensões
int i, j, N = 2;
p = (int**) malloc(N*sizeof(int*));

for (i = 0; i < N; i++) {
    p[i] = (int *)malloc(N*sizeof(int));
    for (j = 0; j < N; j++)
        scanf("%d", &p[i][j]);
}
```

```
for (i = 0; i < N; i++)
    free(p[i]);
free(p);
```

- Assim como os tipos básicos, também é possível fazer a **alocação dinâmica de estruturas (struct)**.
- O tamanho de uma estrutura é sempre **igual ou maior** que a soma dos tamanhos dos seu campos, pois considera como os dados são armazenados na memória dos computadores (alinhamento).
  - Uso do operador unário **sizeof** é obrigatório.

As regras são exatamente as mesmas:

- Podemos fazer a alocação de uma **única estrutura**.
- Podemos fazer a alocação de um **array de estruturas**.

- Para alocar **uma única estrutura**
  - Ponteiro para **struct** receberá o resultado do **malloc()**
  - Utilizamos o operador seta para acessar o conteúdo
  - Usamos **free()** para liberar a memória alocada

```
struct cadastro{
    char nome[50];
    int idade;
};

int main() {
    struct cadastro *cad = (struct cadastro*) malloc(sizeof(struct cadastro));
    strcpy(cad->nome, "Maria");
    cad->idade = 30;

    free(cad);

    return 0;
}
```

- Para alocar **um array de estruturas**
  - Ponteiro para **struct** receberá o resultado do **malloc()**
  - Utilizamos os **colchetes [ ]** para acessar o conteúdo
  - Usamos **free()** para liberar a memória alocada

```
struct cadastro{
    char nome[50];
    int idade;
};

int main(){
    struct cadastro *vcad = (struct cadastro*) malloc(10*sizeof(struct cadastro));

    strcpy(vcad[0].nome, "Maria");
    vcad[0].idade = 30;

    strcpy(vcad[1].nome, "Cecilia");
    vcad[1].idade = 10;

    strcpy(vcad[2].nome, "Ana");
    vcad[2].idade = 10;

    free(vcad);

    return 0;
}
```

Obs.: `vcad[i].nome`  $\leftrightarrow$  `(vcad + i)->nome`